

Reentrancy

Controllo esterno, stato incoerente, CEI e guard

`withdraw()` → `receive()` → `withdraw()` → ...

Contratti giocattolo vulnerabili apposta, solo in locale con Foundry:
account di test, fondi fittizi, nessun fork di protocolli reali.

Baseline: Solidity 0.8.37
OpenZeppelin Contracts 5.x

1 • OBIETTIVI

Cosa saprai fare alla fine della lezione

Definire la reentrancy

Senza ridurla a «una funzione viene chiamata due volte»: single-function, cross-function, cross-contract.

Trovare il punto di cessione

Individuare il momento preciso in cui il contratto cede il controllo, e quali diritti restano spendibili.

Applicare CEI e capire il guard

Riconoscere il pattern interaction prima dell'effect; sapere cosa fa e cosa non fa un ReentrancyGuard.

Riprodurre e poi difendere

Un test Foundry che riproduce la reentrancy su un giocattolo locale, e il test di regressione dopo la correzione.

2 • MODELLO MENTALE

Il guardaroba: il registro aggiornato troppo tardi

LA PROCEDURA

1. controlli il biglietto
2. consegna il cappotto
3. **solo dopo** segni sul registro che è stato ritirato

Tra il punto 2 e il punto 3 il registro dice ancora: **questo cliente ha diritto a un cappotto.**

NEL CONTRATTO

```
Contratto A
| controlla credito = 1 ETH
|--- invia 1 ETH ---> Contratto B
|                                     | callback
|←-- richiama A -----|
| A vede ANCORA credito = 1 ETH
...
| solo alla fine A mette credito = 0
```

```
stato critico ancora valido + controllo ceduto a codice esterno
+ possibilità di rientrare = reentrancy pericolosa
```

LA DOMANDA DA AUDITOR NON È «QUESTA FUNZIONE FA UNA EXTERNAL CALL?»

**Nel momento della external call, quali invarianti sono temporaneamente falsi,
o quali diritti risultano ancora spendibili?**

3.1 · CHE COSA SIGNIFICA «REENTRANT»

Non serve che sia la stessa funzione

Una funzione è **reentrant** quando, mentre una sua invocazione non è ancora terminata, il flusso di esecuzione torna nel contratto originario attraverso una nuova chiamata.

```
A.withdraw()  
  → chiama B  
    → B richiama A.claimReward()  
      → A.claimReward() osserva stato intermedio
```

DOVE PUÒ COMPARIRE

contratti ERC-20 non fidati

callback esplicite

token con hook

receiver ERC-721 / ERC-1155

vault e adapter

oracle e router

proxy e delegate pattern

Qualsiasi interazione con un altro contratto può trasferire il controllo, non soltanto il trasferimento di Ether: la reentrancy non è un problema esclusivo di `call{value: ...}`.

3.2 · IL CALL STACK

Il primo frame resta sospeso, lo storage è lo stesso

```
frame A.withdraw()
  |
  +-- frame B.receive()
        |
        +-- frame A.withdraw()
              |
              +-- ...
```

QUANDO A CHIAMA B

A non «finisce» e poi parte B: il primo frame di A.withdraw resta sospeso in attesa del ritorno di B.

IL PUNTO CHIAVE

La nuova chiamata entra in un **nuovo frame**, ma legge lo **stesso storage persistente** di A. Se il primo frame non ha ancora aggiornato `credits[msg.sender]`, il secondo legge il valore precedente.

3.3 · ATOMICITÀ

Atomico non significa senza stati intermedi

CHE COSA GARANTISCE L'ATOMICITÀ

Al termine, o le modifiche completate vengono committate, oppure un revert propagato annulla le modifiche del relativo percorso di esecuzione.

CHE COSA NON GARANTISCE

Durante l'esecuzione esistono **stati intermedi osservabili** dalle chiamate annidate.

«Nessun altro può intervenire finché la mia transazione non finisce»: intuizione errata.

Un'altra transazione non si inserisce nel mezzo della tua. Una chiamata annidata sì: la reentrancy avviene nella stessa transazione e nello stesso albero di chiamate.

3.4 • LOCALE O STORAGE

Dove vive il valore che stai usando

```
uint256 amount = credit[msg.sender];  
(bool ok,) = msg.sender.call{value: amount}("");  
credit[msg.sender] = 0;
```

`amount`

Variabile locale di quella specifica invocazione: ogni frame ha la sua.

`credit[msg.sender]`

Vive nello storage del contratto: durante la external call vale ancora il vecchio credito, e una nuova invocazione legge proprio quello.

3.5 · IL VERO PROBLEMA

L'invariante è temporaneamente rotto

LA PROMESSA DEL SISTEMA

```
address(this).balance ≥ totalCredits
```

NEL PATTERN VULNERABILE

1. il credito è ancora registrato
2. il contratto invia Ether
3. il saldo ETH diminuisce
4. il credito non è ancora diminuito

`balance < liabilities`

Nel mezzo della funzione, per un istante, il contratto non è più in grado di onorare ciò che il suo storage promette.

La callback non crea il bug: **sfrutta una finestra** in cui il contratto ha già prodotto un effetto esterno ma non ha ancora reso coerente il proprio accounting.

Non solo la stessa funzione, non solo lo stesso contratto

SINGLE-FUNCTION

```
withdraw()  
  → external call  
    → withdraw()
```

La forma didatticamente più semplice.

CROSS-FUNCTION

```
withdraw()  
  → external call  
    → transferCredit()
```

Il callback entra in una funzione diversa che condivide lo stesso stato: un guard solo su `withdraw()` non basta.

CROSS-CONTRACT

```
Vault A  
  → Adapter B  
    → Token/Hook C  
      → richiama A
```

Per questo l'audit richiede un call graph, non una lettura funzione per funzione.

Nella forma cross-function il destinatario può usare, tramite un'altra entry point, un credito che è già in fase di pagamento.

3.9 · CHECKS-EFFECTS-INTERACTIONS

Consumare il diritto prima di cedere il controllo

```
uint256 amount = credit[msg.sender]; // check/input
if (amount == 0) revert NoCredit();   // check

credit[msg.sender] = 0;                // effect

(bool ok,) = msg.sender.call{value: amount}("");
if (!ok) revert TransferFailed();      // interaction
```

DURANTE LA CALLBACK

```
credit[msg.sender] = 0
```

Una seconda invocazione trova già il credito consumato.

Non memorizzare «metti la call per ultima» come formula. Chiediti: **prima di cedere il controllo, ho consumato o invalidato tutti i diritti che questa operazione sta utilizzando?**

3.10 · «E SE LA CALL FALLISCE DOPO?»

Il revert annulla anche l'SSTORE precedente

```
credit[msg.sender] = 0;  
(bool ok,) = msg.sender.call{value: amount}("");  
if (!ok) revert TransferFailed();
```

```
credito = amount  
↓  
credito = 0  
↓  
external call fallisce  
↓  
revert  
↓  
stato della chiamata ripristinato  
↓  
credito torna = amount
```

Il credito non resta perso: è precisamente ciò che rende naturale applicare CEI in Solidity.

3.11 · REENTRANCYGUARD

Una barriera di ingresso, con una nota di versione

NOT_ENTERED → ENTERED → NOT_ENTERED

Se una funzione protetta viene richiamata mentre il guard è già in stato ENTERED, la nuova invocazione viene rifiutata.

```
function withdraw() external nonReentrant {  
    ...  
}
```

IMPORT CORRENTE, OPENZEPPELIN 5.X

```
import {ReentrancyGuard} from  
    "@openzeppelin/contracts/utils/  
    ReentrancyGuard.sol";
```

NOTA DI VERSIONE

Nella documentazione 5.x il guard storage-based è indicato come deprecato in prospettiva della v6, a favore di ReentrancyGuardTransient (EIP-1153). Un auditor verifica la documentazione della **versione realmente usata dal progetto**.

3.12 · CEI E GUARD NON SONO EQUIVALENTI

Due difese diverse, da tenere entrambe

CEI

Protegge la coerenza logica dello stato.

GUARD

Protegge dall'ingresso annidato nelle funzioni protette.

UN GUARD, PERÒ, NON FA TUTTO QUESTO

non corregge accounting sbagliato

non corregge autorizzazioni errate

non protegge funzioni fuori dal locking model

non impedisce effetti pericolosi in contratti terzi

non sostituisce le invarianti

non rende sicuro un protocollo composabile

Conclusione da evitare: «ho messo nonReentrant, quindi non devo più ragionare sull'ordine dello stato».

3.13 · IL CAVEAT DEI MODIFIER

Due funzioni nonReentrant non si chiamano tra loro

```
function withdraw() external nonReentrant {  
    _withdraw(msg.sender);  
}  
  
function _withdraw(address user) private {  
    // logica condivisa  
}
```

OpenZeppelin documenta che le funzioni protette dallo stesso ReentrancyGuard non possono normalmente chiamarsi l'una con l'altra come entry point nonReentrant, perché il lock è già attivo.

La struttura tipica mette la **barriera sull'entry point esterna** e la logica condivisa in una funzione interna, che non tenta di acquisire di nuovo il lock.

Un vault giocattolo, vulnerabile apposta

```
mapping(address user ⇒ uint256 amount) public credit;
uint256 public totalCredits;

function deposit() external payable {
    if (msg.value == 0) revert ZeroDeposit();
    credit[msg.sender] += msg.value;
    totalCredits += msg.value;
}

function withdraw() external {
    uint256 amount = credit[msg.sender];
    if (amount == 0) revert NoCredit();

    (bool ok,) = msg.sender.call{value: amount}("");
    if (!ok) revert TransferFailed();

    credit[msg.sender] = 0;
    totalCredits -= amount;
}
```

QUELLO CHE FA

CHECKS

INTERACTION

EFFECTS

QUELLO CHE DOVREBBE FARE

CHECKS

EFFECTS

INTERACTION

Il bug è piccolo: due righe spostate. Il concetto è profondo.

5.2 • ANALISI DI WITHDRAW()

L'input più importante non è un parametro

Domanda	Risposta
Chi può chiamarla?	Qualunque address con credito, quindi anche un contratto
Input del chiamante	Nessun parametro ABI, ma il chiamante controlla il codice eseguito da msg.sender quando riceve Ether
Valore in ingresso	Nessuno: la funzione invia valore verso l'esterno
Stato letto	credit[msg.sender]
Stato modificato	credit[msg.sender] = 0 e totalCredits, ma solo dopo l'interazione
Dove cede il controllo	msg.sender.call{value: amount}("") — il chiamato esegue codice prima che la call ritorni

L'ASSUNZIONE IMPLICITA E FALSA

«Dopo aver inviato Ether tornerò subito alla riga successiva, senza che nessuno possa usare nuovamente questo credito.»

5.3 · LA TRACCIA CONCRETA

Un diritto che doveva essere consumato una volta sola

SITUAZIONE INIZIALE

```
Vault balance:      5 ETH  
credit[attacker]:   1 ETH  
totalCredits:       5 ETH
```

COSA SUCCEDDE

```
withdraw() primo frame  
  amount = 1 ETH  
  il vault invia 1 ETH  
  → receive() dell'attacker richiama withdraw()  
  
withdraw() secondo frame  
  credit[attacker] è ANCORA 1 ETH  
  il vault invia di nuovo 1 ETH  
  ...
```

Il pattern si ripete finché ci sono fondi e gas, o fino al limite che imponiamo noi nel fixture di laboratorio.

Che cosa dovrebbe garantire il vault

P1 Credito non riutilizzabile

Una volta iniziato il pagamento, nessuna callback deve poter spendere di nuovo quel credito.

P2 Solvibilità contabile

$\text{balance} \geq \text{totalCredits}$; nel modello semplice, senza Ether forzato, vale l'uguaglianza. In audit reale: $\text{assets} \geq \text{liabilities}$.

P4 Conservazione del valore

Senza nuovi depositi: $\text{payout totale dell'utente} \leq \text{credito}$ legittimamente assegnato.

P5 Callback-safe accounting

Ogni stato osservabile durante una external call deve già rappresentare il credito come consumato.

P5 non è una proprietà «alla fine della transazione»: riguarda lo stato in corrispondenza di un **interaction boundary**.

Una post-condition locale può essere vera mentre l'economia è rotta

P3 • WITHDRAWAL SINGOLO

dopo un `withdrawal` riuscito:
`credit[user] = 0`

Sembra la proprietà giusta. Da sola **non basta**.

SUL CONTRATTO VULNERABILE

Alla fine dell'intera transazione il credito risulta comunque zero — anche se il contratto ha già pagato più volte lo stesso diritto.

Una post-condition locale può risultare vera mentre una proprietà economica globale è stata violata.

Tre vault: il rotto, quello con CEI, quello con CEI e guard

```
solidity-security-course/  
├─ foundry.toml  
├─ lib/  
│   └─ forge-std/  
├─ src/  
│   └─ reentrancy/  
│       ├── VulnerableVault.sol  
│       ├── SafeVaultCEI.sol  
│       └─ SafeVaultGuarded.sol  
└─ test/  
    └─ reentrancy/  
        └─ Reentrancy.t.sol
```

Si continua sul progetto Foundry delle lezioni precedenti; per un progetto nuovo:

```
forge init solidity-security-course  
cd solidity-security-course  
forge --version
```

Baseline invariata: **pragma solidity 0.8.37**, bugfix release del 10 settembre 2026.

7.4 · IL FIXTURE CHE PROVOCA LA CALLBACK

Non un exploit: una fixture locale e deterministica

```
contract LocalReentrantReceiver {
    IVault public immutable vault;
    uint256 public callbacks;
    uint256 public immutable maxCallbacks;

    function depositIntoVault() external payable {
        vault.deposit{value: msg.value}();
    }

    function startWithdrawal() external { vault.withdraw(); }

    receive() external payable {
        if (callbacks < maxCallbacks &&
            address(vault).balance ≥ 1 ether) {
            callbacks++;
            vault.withdraw();
        }
    }
}
```

IL DETTAGLIO DA FAR NOTARE

Quando il vault esegue `msg.sender.call{value: amount}("")`, il `msg.sender` è proprio questo receiver: ottiene il controllo e può richiamare il vault.

Il limite **maxCallbacks** è artificiale: serve a mantenere il laboratorio piccolo e deterministico.

7.5 · IL TEST DEL COMPORTAMENTO VULNERABILE

Quattro depositanti onesti, un receiver con 1 ETH di credito

```
function test_ReentrantCallbackBreaksAccounting() public {  
    // quattro utenti onesti depositano 1 ETH ciascuno  
  
    LocalReentrantReceiver receiver =  
        new LocalReentrantReceiver(IVault(address(vault)), 4);  
    receiver.depositIntoVault{value: 1 ether}();  
  
    assertEq(address(vault).balance, 5 ether);  
    assertEq(vault.credit(address(receiver)), 1 ether);  
  
    receiver.startWithdrawal();  
  
    assertGt(address(receiver).balance, 1 ether);  
    assertEq(address(vault).balance, 0);  
    assertGt(vault.totalCredits(), address(vault).balance);  
}
```

PRIMA

vault: 5 ETH
credito receiver: 1 ETH

DOPO

receiver: > 1 ETH
vault: 0 ETH
totalCredits > balance

7.6 • GLI ASSERT GIUSTI

Una proprietà economica, non un numero del fixture

FRAGILE

```
assertEq(receiver.balance, 5 ether);
```

Dipende dai dettagli del fixture: numero di callback, liquidità, limiti scelti.

ROBUSTO

```
assertGt(address(receiver).balance, 1 ether);  
assertGt(vault.totalCredits(),  
         address(vault).balance);
```

Il receiver aveva diritto a 1 ETH: riceverne di più dimostra il riutilizzo del credito.

liabilities > assets

Il contratto non è più in grado di onorare ciò che il suo stesso storage promette.

7.7 • ESECUZIONE E TRACE

La forma annidata è il fenomeno che stiamo studiando

```
forge test \  
  --match-path test/reentrancy/Reentrancy.t.sol \  
  -vv
```

```
forge test \  
  --match-test test_ReentrantCallbackBreaksAccounting \  
  -vvvv
```

```
VulnerableVault.withdraw()  
  → LocalReentrantReceiver.receive()  
    → VulnerableVault.withdraw()  
      → LocalReentrantReceiver.receive()  
        → VulnerableVault.withdraw()  
        ...
```

Le due lezioni precedenti guardavano una call sola. Qui la trace è un **albero**: è il momento in cui la lettura del call graph diventa indispensabile.

8 • TEST NEGATIVI

Il test sequenziale passa anche sul contratto rotto

```
function test_RevertWhen_DepositIsZero() public {
    vm.expectRevert(VulnerableVault.ZeroDeposit.selector);
    vault.deposit{value: 0}();
}

function test_RevertWhen_NoCredit() public {
    vm.expectRevert(VulnerableVault.NoCredit.selector);
    vault.withdraw();
}

function test_RevertWhen_WithdrawnTwiceSequentially() public {
    // deposita, preleva, poi riprova
    vm.expectRevert(VulnerableVault.NoCredit.selector);
    vault.withdraw();
}
```

ATTENZIONE

Il terzo test **passa anche sul contratto vulnerabile**: withdraw #1 finisce, il credito viene azzerato, e la chiamata successiva fallisce.

Il bug non vive tra due transazioni separate. Vive **dentro la prima**, prima che withdraw #1 abbia terminato.

Due righe spostate, una proprietà salvata

```
function withdraw() external {  
    // CHECKS  
    uint256 amount = credit[msg.sender];  
    if (amount == 0) revert NoCredit();  
  
    // EFFECTS  
    credit[msg.sender] = 0;  
    totalCredits -= amount;  
  
    // INTERACTIONS  
    (bool ok,) = msg.sender.call{value: amount}("");  
    if (!ok) revert TransferFailed();  
}
```

DURANTE LA CALLBACK

credit[receiver] = 0

Una nuova withdraw() incontra subito il check e reverte con NoCredit().

deposit() resta identica: il bug non era lì.

9.3 · UN DETTAGLIO SOTTILE

Se la reentrancy reverte, può far fallire tutto il payout

Il primo receiver chiama **`vault.withdraw()`** direttamente nel `receive`. Contro `SafeVaultCEI` quella chiamata reverte con `NoCredit`; se il `revert` non viene catturato, reverte anche `receive()`, la call del vault torna false e il vault reverte `TransferFailed`.

Non è furto: è un comportamento scomodo per il test.

```
receive() external payable {
    if (!attemptedReentry) {
        attemptedReentry = true;

        (bool ok,) = address(vault).call(
            abi.encodeCall(IVault.withdraw, ())
        );

        reentrySucceeded = ok;
    }
}
```

LocalReentrantProbe osserva due cose — il callback è avvenuto? il tentativo di reentry è riuscito? — senza trasformare il fallimento della chiamata reentrante in fallimento del payout principale.

9.4 · IL TEST DI REGRESSIONE

Non «la funzione funziona», ma «lo scenario patologico non rompe più la proprietà»

```
function test_CEI_PreventsReuseOfCredit() public {  
    // due utenti depositano 1 ETH ciascuno  
    LocalReentrantProbe receiver =  
        new LocalReentrantProbe(IVault(address(safe)));  
    receiver.depositIntoVault{value: 1 ether}();  
  
    receiver.startWithdrawal();  
  
    assertTrue(receiver.attemptedReentry());  
    assertFalse(receiver.reentrySucceeded());  
  
    assertEquals(address(receiver).balance, 1 ether);  
    assertEquals(address(safe).balance, 2 ether);  
    assertEquals(address(safe).balance, safe.totalCredits());  
}
```

LA CALLBACK È AVVENUTA

Il test verifica prima che il tentativo ci sia stato: altrimenti non starebbe provando nulla.

Il receiver riceve **esattamente il proprio credito**, e restano coperti i 2 ETH degli altri depositanti.

CEI e guard insieme, non uno al posto dell'altro

```
import {ReentrancyGuard} from
"@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract SafeVaultGuarded is ReentrancyGuard {

    function withdraw() external nonReentrant {
        // CHECKS
        uint256 amount = credit[msg.sender];
        if (amount == 0) revert NoCredit();

        // EFFECTS
        credit[msg.sender] = 0;
        totalCredits -= amount;

        // INTERACTIONS
        (bool ok,) = msg.sender.call{value: amount}("");
        if (!ok) revert TransferFailed();
    }
}
```

INSTALLAZIONE

```
forge install
    OpenZeppelin/openzeppelin-contracts
// remappings.txt
@openzeppelin/contracts/=
    lib/openzeppelin-contracts/contracts/
```

Non abbiamo rimesso gli effects dopo la call «tanto c'è il guard»: stato coerente e barriera.

9.6 · REGRESSIONE CON IL GUARD

Le stesse domande, un secondo strato di difesa

```
function test_GuardAndCEI_BlockReentry() public {
    SafeVaultGuarded safe = new SafeVaultGuarded();
    // u1 deposita 2 ETH

    LocalReentrantProbe receiver =
        new LocalReentrantProbe(IVault(address(safe)));
    receiver.depositIntoVault{value: 1 ether}();
    receiver.startWithdrawal();

    assertTrue(receiver.attemptedReentry());
    assertFalse(receiver.reentrySucceeded());

    assertEquals(address(receiver).balance, 1 ether);
    assertEquals(safe.credit(address(receiver)), 0);
    assertEquals(address(safe).balance, 2 ether);
    assertEquals(safe.totalCredits(), 2 ether);
}
```

CHI BLOCCA CHI

Qui la reentry fallisce già per il lock, prima ancora del check sul credito: le due difese si sovrappongono di proposito.

Gli assert restano gli stessi: se cambia l'implementazione ma la proprietà regge, il test continua a passare.

9.7 · PRIMA E DOPO

Lo stesso callback, tre esiti

VULNERABILE

```
withdraw()
├─ CHECK credito = 1
├─ CALL 1 ETH ─────────┐
│                       │
│           receive()
│           withdraw()
│           credito ANCORA 1
│           CALL ancora
└─ EFFECT credito = 0 ─┘
```

CORRETTO CON CEI

```
withdraw()
├─ CHECK credito = 1
├─ EFFECT credito = 0
├─ CALL 1 ETH ─────────┐
│                       │
│           receive()
│           withdraw()
│           credito = 0
│           REVERT
└─ ritorno ───────────┘
```

CEI + GUARD

```
withdraw() [lock ENTERED]
├─ CHECK
├─ EFFECT
├─ CALL ───────────┐
│                   │
│           receive()
│           withdraw()
│           guard già ENTERED
│           REVERT
└─ unlock ─────────┘
```

9.8 · NON USARE TRANSFER() COME FIX

La sicurezza non può dipendere dal gas che resta al callee

LA VECCHIA RICETTA

```
recipient.transfer(amount);
```

Suggerita perché inoltrava uno stipendio di gas limitato: «al destinatario non resterà abbastanza gas per fare qualcosa di interessante».

I costi degli opcode e le caratteristiche della piattaforma possono evolvere.

COME SI CORREGGE DAVVERO

- external call riconosciuta come tale
- stato coerente prima di cederla
- gestione esplicita del fallimento
- CEI
- guard quando appropriato

Il problema va corretto a livello di logica e invarianti, non sperando che il callee sia artificialmente incapace di eseguire codice.

Interaction boundary anche senza un solo wei

NON SERVE UN TRASFERIMENTO DI ETH

```
externalProtocol.execute(...);  
token.safeTransfer(to, amount);
```

Se quel contratto può richiamarti, c'è un interaction boundary. A seconda dello standard e della catena di chiamate, un'operazione token può condurre a callback o codice esterno.

```
external control flow ≠ soltanto ETH transfer
```

READ-ONLY REENTRANCY · ANTICIPAZIONE

Una callback può osservare il contratto mentre alcune variabili sono state aggiornate e altre no. Anche una funzione **view** può restituire un valore economicamente incoerente durante una fase intermedia, e un altro componente può usarlo per decidere.

Regola: prima di una external call, rendi coerente **l'intero insieme di stato** che altri componenti possono osservare.

Quando trovi nonReentrant, non smettere l'audit

```
function withdraw() external nonReentrant {  
    ...  
}  
  
function claimBonus() external {    // nessun lock  
    ...  
}
```

Se claimBonus() legge o modifica lo stesso stato economico ed è raggiungibile durante la callback, proteggere soltanto withdraw() può non bastare.

LE DOMANDE GIUSTE

- qual è la risorsa protetta dal lock?
- quali entry point toccano quella risorsa?
- quali sono raggiungibili durante una callback?
- il lock copre davvero il dominio di stato corretto?

È la differenza tra **riconoscere un modifier** e **capire una proprietà di sicurezza**.

Dove passa il controllo, e cosa è ancora spendibile

Il perimetro

- **1.** Dove passa il controllo? Segna ogni call esterna, invio di Ether, callback, token hook, receiver hook, chiamata indiretta.
- **2.** Quale stato è ancora «spendibile»? Credit, allowance interne, nonce, claim, reward, quote, flag non ancora consumati.
- **3.** CEI è realmente rispettato? Non l'ordine estetico: *tutti* gli effetti necessari alla coerenza.

Il grafo

- **4.** Quali funzioni sono raggiungibili in callback? Single-function e cross-function.
- **5.** Esistono dipendenze cross-contract? Una callback può modificare un contratto da cui questa funzione legge prezzi o autorizzazioni?
- **6.** Il destinatario è davvero trusted? Anche una dipendenza nota può chiamare codice non noto a valle.

I guard, gli invarianti, i test

Le difese

- **7.** C'è un nonReentrant? Quali entry point protegge? Condividono lo stesso lock? Esistono percorsi non protetti sullo stesso stato?
- **8.** Gli invarianti valgono *durante* l'interaction boundary, non solo nello stato finale?
- **9.** Il fallimento della call è gestito correttamente? Cosa succede se il destinatario reverte?

Le prove

- **10.** Esiste un test con receiver contrattuale? Testare soltanto EOA lascia invisibile il comportamento delle callback.
- **11.** La solvibilità è testata come proprietà globale? Per vault ed escrow: assets e liabilities restano coerenti?
- **12.** La remediation ha un test di regressione? Una vulnerabilità corretta senza test può essere reintrodotta.

Prima la proprietà, poi il test

1 • Ricostruisci la trace

Per tre livelli di chiamata: frame, balance del vault, credito, totalCredits. In quale riga cambia ciascuno?

2 • Trova il bug senza eseguire

claim() che trasferisce il token e poi azzera il reward: dov'è l'interaction boundary? Come riordini?

3 • Cross-function

Aggiungi moveCredit() al vault rotto: la callback può usarlo mentre il credito è ancora stale?

4 • Receiver che reverte

Contro SafeVaultCEI: withdraw reverte, il credito non si perde, totalCredits non resta decrementato. Perché?

5 • Regression test minimo

Il test più piccolo che fallisce su VulnerableVault e passa su SafeVaultCEI, con una proprietà economica.

6 • Threat model

Per SafeVaultGuarded: asset, attori, trust boundary, external call, stato critico, proprietà, fallimenti.

7 • Guard scope

Una seconda funzione che tocca credit: può osservare stato transitorio durante un callback?

8 • Test parametrico

Fuzz semplice sulla solvibilità: credito, liquidità, numero massimo di callback, con bound ragionevoli.

Otto idee, e il flusso di lavoro del corso

1 · Una external call trasferisce controllo

Il chiamato può richiamarti prima che la prima funzione sia terminata.

2 · Reentrancy ≠ «richiamare la stessa funzione»

Single-function, cross-function, cross-contract.

3 · Cerca i diritti ancora validi

Credit, claim, reward, nonce, quote: consumali prima di cedere il controllo.

4 · CEI è un principio di coerenza

Il mondo esterno deve osservare uno stato già consistente.

5 · nonReentrant è una barriera, non una prova

Restano accounting, scope del lock, altre entry point, dipendenze.

6 · Testa contratti come caller, non solo EOA

Molte proprietà emergono solo quando msg.sender può fare callback.

7 · Una post-condition locale può ingannarti

credit[user] == 0 alla fine non esclude payout multipli.

8 · Dopo ogni fix, conserva il controesempio

Diventa il test di regressione.

requisito → proprietà → implementazione → test → **controesempio locale** → remediation → **regression test**

13 · FONTI DELLA LEZIONE

Da dove viene ogni affermazione

Fonte	Per che cosa
Solidity · Security Considerations	modello di reentrancy, trasferimento di controllo, pattern CEI
Solidity 0.8.37 Release Announcement	versione usata negli esempi e correzioni recenti
OpenZeppelin 5.x · Utils / ReentrancyGuard	API, import path, nonReentrant, variante transient
OpenZeppelin 5.x · installazione con Foundry	remapping corrente e uso delle release pubblicate
Foundry · Writing Tests	struttura e convenzioni dei test Forge
Foundry · expectRevert	comportamento corrente dei test negativi con revert
OWASP · SCWE-046 Reentrancy Attacks	classificazione, cause tipiche, remediation
OWASP Top 10 2026 · SC08 Reentrancy	single-function, cross-function, cross-contract

Prima della lezione successiva: guarda una external call e chiediti subito **quale stato può osservare o riutilizzare una callback in questo preciso momento.**